# Упражнения: Отражения на типовете

## 1. Крадец

Добавете към проекта си класа **Hacker** от кутията по-долу.

Hacker.cs

```
public class Hacker
{
    public string username = "securityGod82";
    private string password = "mySuperSecretPassw0rd";

    public string Password
    {
        get => this.password;
        set => this.password = value;
    }

    private int Id { get; set; }

    public double BankAccountBalance { get; private set; }

    public void DownloadAllBankAccountsInTheWorld()
    {
    }
}
```

Има един много гаден хакер, но не е много умен. Опитва се да открадне голяма сума пари и да я прехвърли в собствената си сметка. Полицията го преследва, но им трябва професионалист... Правилно – това сте вие!

Разполагате с информацията, че този хакер пази част от информацията си в private полета. Създайте нов клас, наречен Spy, и добавете в него метод, наречен StealFieldInfo, който да получава:

- низ – име на класа, който да разследва
- масив от низове – имена на полетата, които да разследва

След като намерите полетата, отпечатайте на конзолата:

- "Class under investigation: {имеНаКласа}"

На следващите редове отпечатайте информация за всяко поле в настоящия формат:

- "{имеНаПоле} = {стойностНаПоле}"

Използвайте StringBuilder да свържете отговора. Не променяйте нищо в "Hacker" класа!

В Main метод трябва да можете да проверите програмата си с това парче код:

Документът е разработен за нуждите на Национална програма "Обучение за ИТ умения и кариера" на Министерството на образованието и науката (МОН) и се разпространява под свободен лиценз CC-BY-NC-SA (Creative Commons Attribution-Non-Commercial-Share-Alike 4.0 International).

```
public static void Main()
{
    Spy spy = new Spy();
    string result = spy.StealFieldInfo("Hacker", "username", "password");
    Console.WriteLine(result);
}
```

## Пример

| Изход |
| --- |
| Class under investigation: Hacker<br>username = securityGod82<br>password = mySuperSecretPassw0rd |

## Решение

```
public string StealFieldInfo(string investigatedClass, params string[] requestedFields)
{
    Type classType = Type.GetType(investigatedClass);
    FieldInfo[] classFields = classType.GetFields(
        BindingFlags.Instance | BindingFlags.Static | BindingFlags.NonPublic | BindingFlags.Public);
    StringBuilder stringBuilder = new StringBuilder();

    Object classInstance = Activator.CreateInstance(classType, new object[] { });

    stringBuilder.AppendLine($"Class under investigation: {investigatedClass}");

    foreach (FieldInfo field in classFields.Where(f => requestedFields.Contains(f.Name)))
    {
        stringBuilder.AppendLine($"{field.Name} = {field.GetValue(classInstance)}");
    }

    return stringBuilder.ToString().Trim();
}
```

## Висококачествени грешки

Вече сте експерт в Качествен Код, така че знаете какъв вид модификатори на достъпа трябва да бъде зададен на членовете на класа. Трябва да сте забелязали, че нашият хакер не е запознат с тези концепции.

Създайте метод в своя Spy клас, наречен AnalyzeAcessModifiers(string className). Проверете всички модификатори на достъп на полетата и методите. Отпечатайте на конзолата всички грешки във формат:

## Полета

- {имеНаПоле} must be private!

## Getters

- {имеНаМетод} have to be public!

## Setters

- {имеНаМетод} have to be private!

Използвайте StringBuilder да свържете отговора. Не променяйте нищо в "Hacker" класа!

Документът е разработен за нуждите на Национална програма "Обучение за ИТ умения и кариера" на Министерството на образованието и науката (МОН) и се разпространява под свободен лиценз CC-BY-NC-SA (Creative Commons Attribution-Non-Commercial-Share-Alike 4.0 International).

В Main метода си би трябвало да можете да проверите програмата си със следния блок от код:

```
public static void Main()
{
    Spy spy = new Spy();
    string result = spy.AnalyzeAcessModifiers("Hacker");
    Console.WriteLine(result);
}
```

## Пример

| Изход |
| --- |
| username must be private!<br>get_Id have to be public!<br>set_Password have to be private! |

## Решение

```
public string AnalyzeAcessModifiers(string investigatedClass)
{
    Type classType = Type.GetType(investigatedClass);
    FieldInfo[] classFields = classType.GetFields(BindingFlags.Instance | BindingFlags.Static | BindingFlags.Public);
    MethodInfo[] classPublicMethods = classType.GetMethods(BindingFlags.Instance | BindingFlags.Public);
    MethodInfo[] classNonPublicMethods = classType.GetMethods(BindingFlags.Instance | BindingFlags.NonPublic);

    StringBuilder stringBuilder = new StringBuilder();

    foreach (FieldInfo field in classFields)
    {
        stringBuilder.AppendLine($"{field.Name} must be private!");
    }
    foreach (MethodInfo method in classNonPublicMethods.Where(m => m.Name.StartsWith("get")))
    {
        stringBuilder.AppendLine($"{method.Name} have to be public!");
    }
    foreach (MethodInfo method in classPublicMethods.Where(m => m.Name.StartsWith("set")))
    {
        stringBuilder.AppendLine($"{method.Name} have to be private!");
    }

    return stringBuilder.ToString().Trim();
}
```

## Мисия „Частно“ невъзможна

Време е да видим какво цели този хакер, с който се разправяме. Създайте метод в своя Spy клас, наречен RevealPrivateMethods(stirng className). Отпечатайте всички частни методи в следния формат:

- All Private Methods of Class: {имеНаКлас}
- Base Class: {базовКлас}

На следващите редове отпечатайте имената на намерените методи, всеки на нов ред.

Използвайте StringBuilder да свържете отговора. Не променяйте нищо в "Hacker" класа!

В метода Main трябва да можете да проверите програмата си със следния код:

Документът е разработен за нуждите на Национална програма "Обучение за ИТ умения и кариера" на Министерството на образованието и науката (МОН) и се разпространява под свободен лиценз CC-BY-NC-SA (Creative Commons Attribution-Non-Commercial-Share-Alike 4.0 International).

```
public static void Main()
{
    Spy spy = new Spy();
    string result = spy.RevealPrivateMethods("Hacker");
    Console.WriteLine(result);
}
```

## Пример

| Изход |
| --- |
| All Private Methods of Class: Hacker<br>Base Class: Object<br>get_Id<br>set_Id<br>set_BankAccountBalance<br>Finalize<br>MemberwiseClone |

## Решение

```
public string RevealPrivateMethods(string investigatedClass)
{
    Type classType = Type.GetType(investigatedClass);
    MethodInfo[] classMethods = classType.GetMethods(BindingFlags.Instance | BindingFlags.NonPublic);
    StringBuilder stringBuilder = new StringBuilder();

    stringBuilder.AppendLine($"All Private Methods of Class: {investigatedClass}");
    stringBuilder.AppendLine($"Base Class: {classType.BaseType.Name}");

    foreach (MethodInfo method in classMethods)
    {
        stringBuilder.AppendLine(method.Name);
    }

    return stringBuilder.ToString().Trim();
}
```

## Колектор

Използвайте отражение, за да уловите всички "Hacker" методи. След това подгответе алгоритъм, който да разпознае кои методи са getters и setters.

Отпечатайте на конзолата всеки getter на нов ред във формат:

- {име} will return {Връщан Тип}

След това отпечатайте всички setters във формат:

- {име} will set field of {Тип на параметър}

Използвайте StringBuilder да свържете отговора. Не променяйте нищо в "Hacker" класа!

В Main метода трябва да можете да проверите програмата си със следните няколко реда:

Документът е разработен за нуждите на Национална програма "Обучение за ИТ умения и кариера" на Министерството на образованието и науката (МОН) и се разпространява под свободен лиценз CC-BY-NC-SA (Creative Commons Attribution-Non-Commercial-Share-Alike 4.0 International).

```
public static void Main()
{
    Spy spy = new Spy();
    string result = spy.CollectGettersAndSetters("Hacker");
    Console.WriteLine(result);
}
```

### Пример

| Изход |
| --- |
| get_Password will return System.String<br>get_Id will return System.Int32<br>get_BankAccountBalance will return System.Double<br>set_Password will set field of System.String<br>set_Id will set field of System.Int32<br>set_BankAccountBalance will set field of System.Double |

### Решение

```
public string CollectGettersAndSetters(string investigatedClass)
{
    Type classType = Type.GetType(investigatedClass);

    MethodInfo[] classMethods =
        classType.GetMethods(BindingFlags.Instance | BindingFlags.NonPublic | BindingFlags.Public);
    StringBuilder stringBuilder = new StringBuilder();

    foreach (MethodInfo method in classMethods.Where(m => m.Name.StartsWith("get")))
    {
        stringBuilder.AppendLine($"{method.Name} will return {method.ReturnType}");
    }

    foreach (MethodInfo method in classMethods.Where(m => m.Name.StartsWith("set")))
    {
        stringBuilder.AppendLine($"{method.Name} will set field of {method.GetParameters().First().ParameterType}");
    }

    return stringBuilder.ToString().Trim();
}
```

## 2. Поля за жътва

Даден ви е клас **RichSoilLand** с много полета (вижте предоставената конструкция). Като добър фермер, какъвто вие сте, трябва да ожънете тези полета. Да ги ожънете означава, че трябва да отпечатате всяко поле в определен формат (както е в изхода).

### Вход

Ще получите максимум 100 реда с една от следните команди:

- private – отпечатва всички private полета
- protected – отпечатва всички protected полета
- public – отпечатва всички public полета
- all – отпечатва ВСИЧКИ декларирани полета
- HARVEST – край на входните данни

### Изход

За всяка команда трябва да отпечатате полетата, които имат съответния модификатор за достъп, описан във входната секция. Форматът, в който полетата трябва да се отпечатат, е:

"<access modifier> <field type> <field name>"

## Примери

| Вход | Изход |
|---|---|
| protected<br>HARVEST | protected String testString<br>protected Double aDouble<br>protected Byte testByte<br>protected StringBuilder aBuffer<br>protected BigInteger testBigNumber<br>protected Single testFloat<br>protected Object testPredicate<br>protected Object fatherMotherObject<br>protected String moarString<br>protected Exception inheritableException<br>protected Stream moarStreamz |
| private<br>public<br>private<br>HARVEST | private Int32 testInt<br>private Int64 testLong<br>private Calendar aCalendar<br>private Char testChar<br>private BigInteger testBigInt<br>private Thread aThread<br>private Object aPredicate<br>private Object hiddenObject<br>private String anotherString<br>private Exception internalException<br>private Stream secretStream<br>public Double testDouble<br>public String aString<br>public StringBuilder aBuilder<br>public Int16 testShort<br>public Byte aByte<br>public Single aFloat<br>public Thread testThread<br>public Object anObject<br>public Int32 anotherIntBitesTheDust<br>public Exception justException<br>public Stream aStream<br>private Int32 testInt<br>private Int64 testLong<br>private Calendar aCalendar<br>private Char testChar<br>private BigInteger testBigInt<br>private Thread aThread<br>private Object aPredicate<br>private Object hiddenObject<br>private String anotherString<br>private Exception internalException<br>private Stream secretStream |
| all<br>HARVEST | private Int32 testInt<br>public Double testDouble<br>protected String testString |

Документът е разработен за нуждите на Национална програма "Обучение за ИТ умения и кариера" на Министерството на образованието и науката (МОН) и се разпространява под свободен лиценз CC-BY-NC-SA (Creative Commons Attribution-Non-Commercial-Share-Alike 4.0 International).

| | |
|---|---|
| | private Int64 testLong<br>protected Double aDouble<br>public String aString<br>private Calendar aCalendar<br>public StringBuilder aBuilder<br>private Char testChar<br>public Int16 testShort<br>protected Byte testByte<br>public Byte aByte<br>protected StringBuilder aBuffer<br>private BigInteger testBigInt<br>protected BigInteger testBigNumber<br>protected Single testFloat<br>public Single aFloat<br>private Thread aThread<br>public Thread testThread<br>private Object aPredicate<br>protected Object testPredicate<br>public Object anObject<br>private Object hiddenObject<br>protected Object fatherMotherObject<br>private String anotherString<br>protected String moarString<br>public Int32 anotherIntBitesTheDust<br>private Exception internalException<br>protected Exception inheritableException<br>public Exception justException<br>public Stream aStream<br>protected Stream moarStreamz<br>private Stream secretStream |

## 3. Black Box Integer

Помагате на свой приятел, който е все още в OOP Basics курса – името му е Пешослав (да не се бърка с реални хора или треньори). Той е малко „бавничък“ и е направил клас, в който всички членове са private. Вашите задачи са да представите с конкретни примери обект от неговия клас (винаги с начална стойност 0) и после да извикате всички методи, които той има. Ограничението ви е да не променяте „ръчно“ нищо в самия клас (смятайте го за черна кутия). Можете да разглеждате класа му, но не го пипайте! Самият клас се казва BlackBoxInt и е обвивка за базовия тип int.

Методите, които има този клас, са:

- Add(int)
- Subtract(int)
- Multiply(int)
- Divide(int)
- LeftShift(int)
- RightShift(int)

### Вход

Входът ще се състои от редове във вида:

- <име на метод>_<стойност>

Например: Add_115

Входът винаги ще е валиден и в описания формат, така че няма нужда да го проверявате изрично. Спирате да получавате вход когато срещнете командата "END".

### Изход

Всяка команда (освен END) трябва да отпечатва настоящата стойност на innerValue от BlackBoxInt обекта, който представяте. Не мамете, като предефинирате ToString() в класа – трябва да вземете стойността от private полето.

### Примери

| Вход | Изход |
|---|---|
| Add_999999<br>Subtract_19<br>Divide_4<br>Multiply_2<br>RightShift_1<br>LeftShift_3<br>END | 999999<br>999980<br>249995<br>499990<br>249995<br>1999960 |

## 4. BarracksWars – Нова фабрика

Даден ви е малък конзолен проект, наречен Barracks (неговият код е включен в предоставената програмна конструкция).

Основните функционалности на проекта са добавяне на нови единици към склада и отпечатване на доклад със статистики за единиците, които в момента са в склада. Първо нека прегледаме оригиналната задача преди проектът да е бил създаден:

### Вход

Входът се състои от команди, всяка на отделен ред. Командите, които изпълняват функционалностите, са:

- add <Archer/Swordsman/Pikeman/{...}> - добавя единица към склада.
- report – отпечатва статистика по лексикологичен ред за единиците в склада.
- fight – край на входните данни.

### Изход

Всяка команда освен fight трябва да отпечатва изхода си на конзолата.

add трябва да отпечатва: "<Archer/Swordsman/Pikeman/{...}> added!"

report трябва да отпечатва цялата информация в склада във формата: "<UnitType> -> <UnitQuantity>", сортиран с UnitType

### Ограничения

Входът ще представлява не повече от 1000 реда

Командата report никога няма да бъде дадена преди коя да е валидна команда да е дадена

### Вашата задача

1) Трябва да проучите кода на проекта и да разберете как работи. В него обаче има части, които не са имплементирани (оставени са с TODO). Трябва да имплементирате функционалността на метода CreateUnit в класа UnitFactory, така че да създаде единица на базата на нейния тип, получен като параметър. Имплементирайте я по такъв начин, че когато добавите нова единица, тя да

Документът е разработен за нуждите на Национална програма "Обучение за ИТ умения и кариера" на Министерството на образованието и науката (МОН) и се разпространява под свободен лиценз CC-BY-NC-SA (Creative Commons Attribution-Non-Commercial-Share-Alike 4.0 International).

може да бъде създадена без да е необходимо да се променя нещо в UnitFactory класа (ще ви кажа на ушенце: използвайте отражение). Може да използвате подхода, наречен Simple Factory.

2) Добавете два нови класа за единици (ще има тестове, които ги изискват) - Horseman с 50 здраве И 10 атака и Gunner с 20 здраве и 20 атака.

Ако правилно изпълните всичко в тази задача, трябва да добавяте код само в папките Factories и Units.

### Примери

| Вход | Изход |
|---|---|
| add Swordsman<br>add Archer<br>add Pikeman<br>report<br>add Pikeman<br>add Pikeman<br>report<br>fight | Swordsman added!<br>Archer added!<br>Pikeman added!<br>Archer -> 1<br>Pikeman -> 1<br>Swordsman -> 1<br>Pikeman added!<br>Pikeman added!<br>Archer -> 1<br>Pikeman -> 3<br>Swordsman -> 1 |
| add Pikeman<br>add Pikeman<br>add Gunner<br>add Horseman<br>add Archer<br>add Gunner<br>add Gunner<br>add Horseman<br>report<br>fight | Pikeman added!<br>Pikeman added!<br>Gunner added!<br>Horseman added!<br>Archer added!<br>Gunner added!<br>Gunner added!<br>Horseman added!<br>Archer -> 1<br>Gunner -> 3<br>Horseman -> 2<br>Pikeman -> 2 |

## 5. BarracksWars – Командите отвръщат на удара

Както може би сте забелязали, командите в проекта от задача 3 са имплементирани чрез a switch case и извиквания на методи в класа Engine. Въпреки че този подход работи, той има недостатъци когато добавяте нова команда, защото за нея трябва да бъде добавен нов case. В някои проекти може да нямате достъп до класа Engine и това не би работило. Представете си този проект да е outsource-нат - outsourcing фирмата няма да има достъп до двигателя. Направете така, че когато искат да добавят нова команда, нищо в Engine да не трябва да се променя.

За да постигнете това, употребете шаблон в проектирането, наречен Command Pattern. Правили сме това и преди в BashSoft Lab и можете да погледнете и там за съвети. Използвайте предоставеният интерфейс Executable като рамка за класовете на командите. Поставете новите командни класове в предоставения commands пакет в core. Също може да направите интерпретатор на команди, за да откачите тази функционалност от Engine. Ето как трябва да изглежда базовата (абстрактна) команда:

Документът е разработен за нуждите на Национална програма "Обучение за ИТ умения и кариера" на Министерството на образованието и науката (МОН) и се разпространява под свободен лиценз CC-BY-NC-SA (Creative Commons Attribution-Non-Commercial-Share-Alike 4.0 International).

```
public abstract class Command : IExecutable
{
    private string[] data;
    private IRepository repository;
    private IUnitFactory unitFactory;

    protected Command(string[] data, IRepository repository, IUnitFactory unitFactory)
    {
        this.Data = data;
        this.Repository = repository;
        this.UnitFactory = unitFactory;
    }

    protected string[] Data
    {
        get { return this.data; }
        private set { this.data = value; }
    }

    protected IRepository Repository
    {
        get{ return this.repository; }
        private set{ this.repository = value; }
    }

    protected IUnitFactory UnitFactory
    {
        get { return this.unitFactory; }
        private set { this.unitFactory = value; }
    }

    public abstract string Execute();
}
```

Забележете, че всички команди, които разширяват тази, ще имат както склад, така и UnitFactory, въпреки че не всички се нуждаят от тях. Оставете това така за тази задача, защото за да работи отражението трябва всички конструктори да приемат едни и същи параметри. Ще видим как да заобиколим този проблем в следващата задача.

След като сте имплементирали шаблона, добавете нова команда. Ще има следният синтаксис:

retire <UnitType> - Всичко, което трябва да направи, е да премахне единица от дадения тип от склада.

Ако в момента няма такива единици в склада, отпечатайте: "No such units in repository."

Ако в момента има такъв единици в склада, отпечатайте: "<UnitType> retired!"

За да имплементирате командата, ще трябва да имплементирате и съответния метод в UnitRepository.

Ако правилно изпълните всичко в тази задача, трябва да пишете/променяте код само в Core и Data пакетите.

Документът е разработен за нуждите на Национална програма "Обучение за ИТ умения и кариера" на Министерството на образованието и науката (МОН) и се разпространява под свободен лиценз CC-BY-NC-SA (Creative Commons Attribution-Non-Commercial-Share-Alike 4.0 International).

## Примери

| Вход | Изход |
|---|---|
| retire Archer<br>add Pikeman<br>add Pikeman<br>add Gunner<br>add Horseman<br>add Archer<br>add Gunner<br>add Gunner<br>add Horseman<br>report<br>retire Gunner<br>retire Archer<br>report<br>retire Swordsman<br>retire Archer<br>fight | No such units in repository.<br>Pikeman added!<br>Pikeman added!<br>Gunner added!<br>Horseman added!<br>Archer added!<br>Gunner added!<br>Gunner added!<br>Horseman added!<br>Archer -> 1<br>Gunner -> 3<br>Horseman -> 2<br>Pikeman -> 2<br>Gunner retired!<br>Archer retired!<br>Archer -> 0<br>Gunner -> 2<br>Horseman -> 2<br>Pikeman -> 2<br>No such units in repository.<br>No such units in repository. |

## 6. * BarracksWars - Завръщане на Зависимостите

В последната част от тази епична трилогия от задачи ще разрешим проблема, при който всички команди получават всички utility класове като параметри в своите конструктори. Можем да постигнем това, използвайки подход, наречен dependency injection container. Този подход се използва в много библиотеки.

Ще променим малко този подход. Премахнете всички полета от абстрактните команди освен data. Вместо това сложете каквито полета са нужни на всяка команда в конкретния клас. Създайте параметър, наречен Inject, и направете така, че да може да се използва само на полета. Подайте този параметър за полетата, които трябва да променим чрез отражение. След като сте подготвили всичко това, напишете необходимия код за отражение в Командния Интерпретатор (който трябва да сте преработили от Engine в предната задача).

Можете да използвате същия пример като в предната задача, за да проверите дали сте я изпълнили правилно.

Документът е разработен за нуждите на Национална програма "Обучение за ИТ умения и кариера" на Министерството на образованието и науката (МОН) и се разпространява под свободен лиценз CC-BY-NC-SA (Creative Commons Attribution-Non-Commercial-Share-Alike 4.0 International).